

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Операционные среды и системное программирование

ОТЧЕТ
к лабораторной работе №2
на тему

**ОБРАБОТКА ТЕКСТОВОЙ ИНФОРМАЦИИ.
РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ**

Студент
Преподаватель

Е. С. Кахновский
Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Теоретические сведения	4
3 Результат выполнения	5
Заключение	5
Список использованных источников	7
Приложение А (обязательное) Листинг кода	8

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение методов и средств обработки текстовой информации, включая регулярные выражения, и использующих их утилит.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Одним из принципов *Unix*-систем является широкое использование текстовых данных: конфигурационные файлы, входные и выходные данные программ в **nix* часто организованы в виде обычного текста. Регулярные выражения – это мощный инструмент для манипуляции текстовой информацией.

Регулярные выражения – это специальным образом записанные строки, используемые для поиска символьных шаблонов в тексте. Многие утилиты для работы с текстом в *Linux* и языки программирования включают в себя механизм регулярных выражений. Здесь возникают проблемы: разные программы и языки оперируют различными диалектами регулярных выражений [1].

Программа *grep* – это основной инструмент для работы с регулярными выражениями. *Grep* анализирует данные со стандартного ввода, ищет совпадения с указанным шаблоном и выводит все подходящие строки. Обычно *grep* предустановлен в большинстве дистрибутивов. *Grep* имеет следующий синтаксис: *grep [параметры] регулярное_выражение [файл...]*. Самый простой случай использования *grep* – поиск строк, содержащих фиксированную подстроку. У *grep* имеются множество параметров, подробно с ними ознакомиться можно в документации [2].

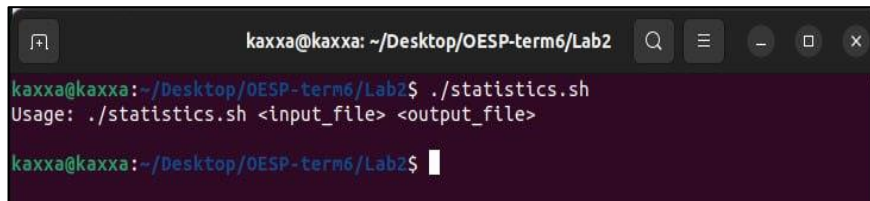
Потоковый текстовый редактор «*sed*» встроен в *bash Linux Ubuntu*. Он использует построчное чтение, а также позволяет выполнить фильтрацию и преобразование текста.

AWK – это скриптовый язык, который полезен при работе в командной строке и широко применяется для обработки текста. При использовании *awk* можно выбирать данные – один или более отдельных фрагментов текста – на основе заданного критерия. Например, с помощью *awk* можно выполнять поиск конкретного слова или шаблона во фрагменте текста, а также выбирать определённую строку/столбец в файле.

Простейшая форма команды *awk* подразумевает описание основного действия в одинарных кавычках и фигурных скобках с указанием после него целевого файла. Выглядеть она может так: *awk '{action}' filename.txt*. Когда нужно найти текст, соответствующий конкретному шаблону, или же конкретное слово в тексте, команда принимает следующий вид: *awk '/regexpattern/{action}' filename.txt* [3].

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

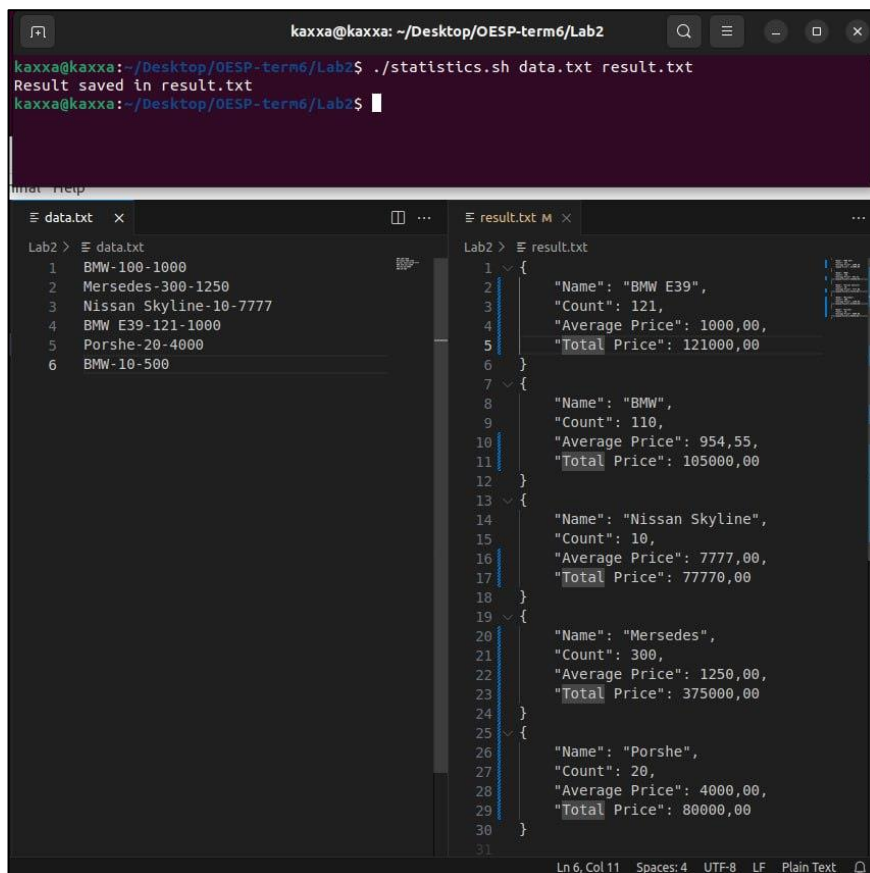
В результате лабораторной работы была создана консольная программа, осуществляющая сбор статистики из данных, находящихся в указанном файле, код которой представлен в приложении А. При запуске программы без передачи аргументов появляется информация о возможных параметрах командной строки (рисунок 1).



```
kaxxa@kaxxa: ~/Desktop/OESP-term6/Lab2
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab2$ ./statistics.sh
Usage: ./statistics.sh <input_file> <output_file>
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab2$
```

Рисунок 1 – Вывод информации о параметрах

Скрипт считывает данные из файла в формате «Имя/Количество/Цена», далее он собирает статистику и выводит результат в заданный результирующий файл (рисунок 2).



```
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab2$ ./statistics.sh data.txt result.txt
Result saved in result.txt
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab2$
```


File	Line	Content
data.txt	1	BMW-100-1000
	2	Mercedes-300-1250
	3	Nissan Skyline-10-7777
	4	BMW E39-121-1000
	5	Porsche-20-4000
	6	BMW-10-500
result.txt	1	{
	2	"Name": "BMW E39",
	3	"Count": 121,
	4	"Average Price": 1000,00,
	5	"Total Price": 121000,00
	6	}
	7	{
	8	"Name": "BMW",
	9	"Count": 110,
	10	"Average Price": 954,55,
	11	"Total Price": 105000,00
12	}	
13	{	
14	"Name": "Nissan Skyline",	
15	"Count": 10,	
16	"Average Price": 7777,00,	
17	"Total Price": 77770,00	
18	}	
19	{	
20	"Name": "Mercedes",	
21	"Count": 300,	
22	"Average Price": 1250,00,	
23	"Total Price": 375000,00	
24	}	
25	{	
26	"Name": "Porsche",	
27	"Count": 20,	
28	"Average Price": 4000,00,	
29	"Total Price": 80000,00	
30	}	
31	}	

Рисунок 2 – Результат работы скрипта

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены методы и средства обработки текстовой информации, включая регулярные выражения, и использование соответствующих утилит.

Практический результат работы – создание *bash* скрипта, обеспечивающего сбор статистики по данным, которые считываются из указанного файла, и записываются в результирующий файл.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Регулярные выражения Bash [Электронный ресурс]. – Режим доступа: <https://timeweb.cloud/tutorials/linux/regulyarnye-vyrazheniya-bash-gajd>.
- [2] Основные приёмы обработки строк в bash [Электронный ресурс]. – Режим доступа: <https://eternalhost.net/base/vps-vds/bash-rabota-so-strokami>
- [3] Команда awk – примеры использования в Linux и Unix [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/ruvds/articles/665084/>.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – Код программы

```
#!/bin/bash

# Check parametrs in cmd line
if [ $# -ne 2 ]; then
    echo "Usage: $0 <input_file> <output_file>"
    echo $help_info
    exit 1
fi

# Params
input_file=$1
output_file=$2

# Check that file exist
if [ ! -f "$input_file" ]; then
    echo "File $input_file not found!"
    exit 1
fi

# Create outpet file, if doesn't exist
> "$output_file"

awk -F- '
{
    items[$1]["count"] += $2;
    items[$1]["total_price"] += $2*$3;
}
END {
    for (item in items) {
        avg_price = items[item]["count"] > 0 ? items[item]["total_price"]
/ items[item]["count"] : 0

        printf("{\n    \"Name\": \"%s\", \n", item);
        printf("    \"Count\": %d, \n", items[item]["count"]);
        printf("    \"Average Price\": %.2f, \n", avg_price);
        printf("    \"Total Price\": %.2f\n}\n",
items[item]["total_price"]);

    }
}' "$input_file" >> "$output_file"

echo "Result saved in $output_file"
```